

Lecture 6

Quicksort



Slides are based on the textbook and its notes

Overview

- ❑ We start **Part II *Sorting and Order Statistics*** from this lecture, in this part, the following topics will be discussed:
 - Quicksort (Lecture 6/Chapter 7)
 - Medians and Order Statistics (Lecture 7/Chapter 9)
 - Heapsort (Lecture 8/Chapter 6)
 - Sorting in Linear Time (Lecture 9/Chapter 8)
- ❑ Quicksort
 - Worst-case running time is $O(n^2)$.
 - Randomized version has expected running time $\Theta(n \lg n)$, assuming that all elements to be sorted are distinct.
 - Constants hidden in $\Theta(n \lg n)$ are small.
 - Sorts in place.

Description of quicksort

- *Quicksort* is based on the three-step process of divide-and-conquer.
 - To sort the subarray $A[p : r]$:
 - Divide:** Partition $A[p : r]$, into two (possibly empty) subarrays $A[p : q - 1]$ and $A[q + 1 : r]$, such that each element in the first subarray $A[p : q - 1]$ is $\leq A[q]$ and $A[q]$ is $\leq$ each element in the second subarray $A[q + 1 : r]$.
 - Conquer:** Sort the two subarrays by recursive calls to QUICKSORT.
 - Combine:** No work is needed to combine the subarrays, because they are sorted in place.
 - Perform the divide step by a procedure PARTITION, which returns the index q that marks the position separating the subarrays.

QUICKSORT(A, p, r)

```
1  if  $p < r$ 
2      // Partition the subarray around the pivot, which ends up in  $A[q]$ .
3       $q = \text{PARTITION}(A, p, r)$ 
4      QUICKSORT( $A, p, q - 1$ ) // recursively sort the low side
5      QUICKSORT( $A, q + 1, r$ ) // recursively sort the high side
```

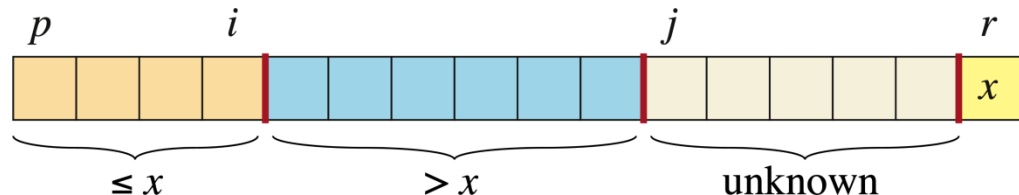
Partitioning

- Partition subarray $A[p : r]$ by the following procedure:

PARTITION(A, p, r)

```
1   $x = A[r]$  // the pivot
2   $i = p - 1$  // highest index into the low side
3  for  $j = p$  to  $r - 1$  // process each element other than the pivot
4      if  $A[j] \leq x$  // does this element belong on the low side?
5           $i = i + 1$  // index of a new slot in the low side
6          exchange  $A[i]$  with  $A[j]$  // put this element there
7  exchange  $A[i + 1]$  with  $A[r]$  // pivot goes just to the right of the low side
8  return  $i + 1$  // new index of the pivot
```

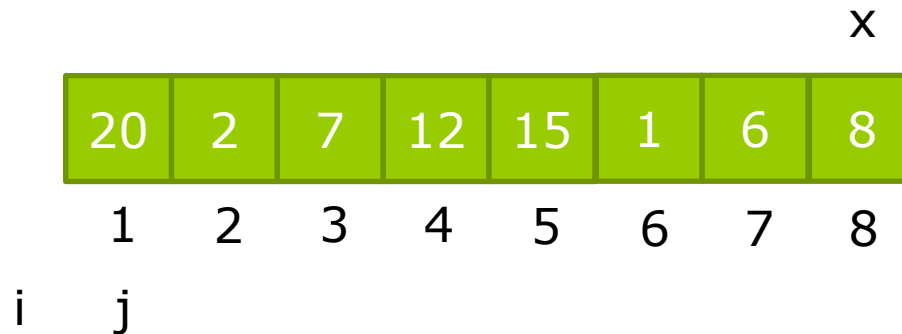
- PARTITION always selects the last element $A[r]$ in the subarray $A[p : r]$ as the *pivot* – the element around which to partition.
- As the procedure executes, the array is partitioned into four regions, some of which may be empty.



Partitioning

PARTITION(A, p, r)

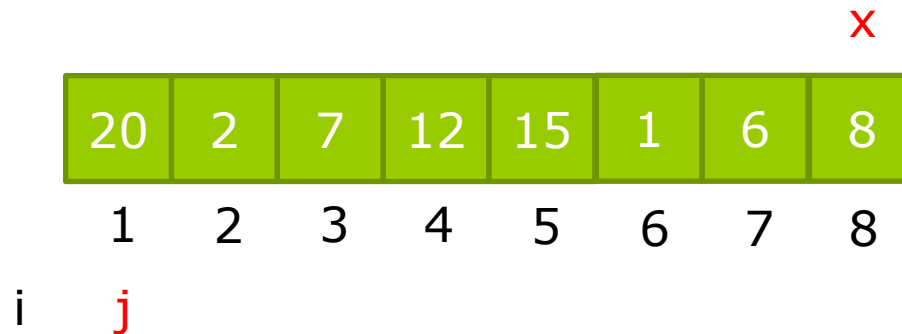
```
1   $x = A[r]$  // the pivot
2   $i = p - 1$  // highest index into the low side
3  for  $j = p$  to  $r - 1$  // process each element other than the pivot
4      if  $A[j] \leq x$  // does this element belong on the low side?
5           $i = i + 1$  // index of a new slot in the low side
6          exchange  $A[i]$  with  $A[j]$  // put this element there
7  exchange  $A[i + 1]$  with  $A[r]$  // pivot goes just to the right of the low side
8  return  $i + 1$  // new index of the pivot
```



Partitioning

PARTITION(A, p, r)

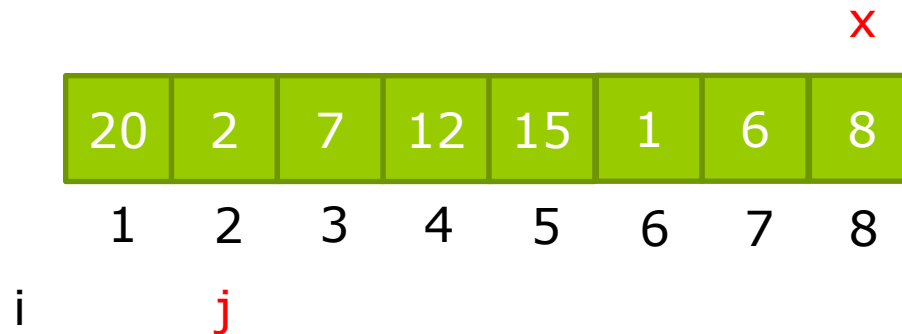
```
1   $x = A[r]$  // the pivot
2   $i = p - 1$  // highest index into the low side
3  for  $j = p$  to  $r - 1$  // process each element other than the pivot
4      if  $A[j] \leq x$  // does this element belong on the low side?
5           $i = i + 1$  // index of a new slot in the low side
6          exchange  $A[i]$  with  $A[j]$  // put this element there
7  exchange  $A[i + 1]$  with  $A[r]$  // pivot goes just to the right of the low side
8  return  $i + 1$  // new index of the pivot
```



Partitioning

PARTITION(A, p, r)

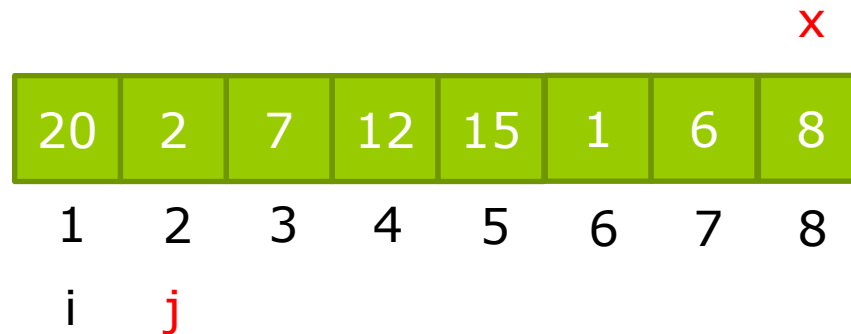
```
1   $x = A[r]$  // the pivot
2   $i = p - 1$  // highest index into the low side
3  for  $j = p$  to  $r - 1$  // process each element other than the pivot
4      if  $A[j] \leq x$  // does this element belong on the low side?
5           $i = i + 1$  // index of a new slot in the low side
6          exchange  $A[i]$  with  $A[j]$  // put this element there
7  exchange  $A[i + 1]$  with  $A[r]$  // pivot goes just to the right of the low side
8  return  $i + 1$  // new index of the pivot
```



Partitioning

PARTITION(A, p, r)

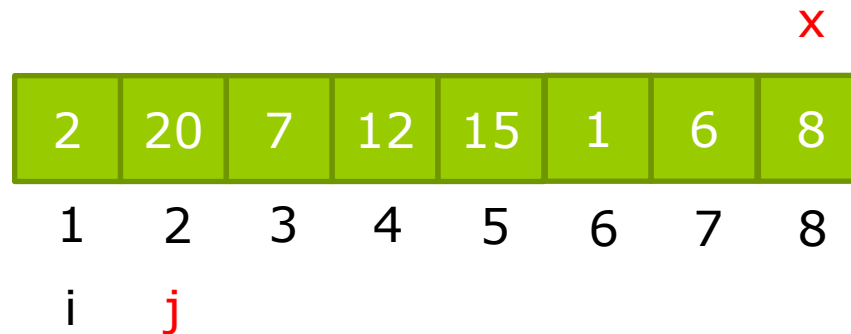
```
1   $x = A[r]$  // the pivot
2   $i = p - 1$  // highest index into the low side
3  for  $j = p$  to  $r - 1$  // process each element other than the pivot
4      if  $A[j] \leq x$  // does this element belong on the low side?
5           $i = i + 1$  // index of a new slot in the low side
6          exchange  $A[i]$  with  $A[j]$  // put this element there
7  exchange  $A[i + 1]$  with  $A[r]$  // pivot goes just to the right of the low side
8  return  $i + 1$  // new index of the pivot
```



Partitioning

PARTITION(A, p, r)

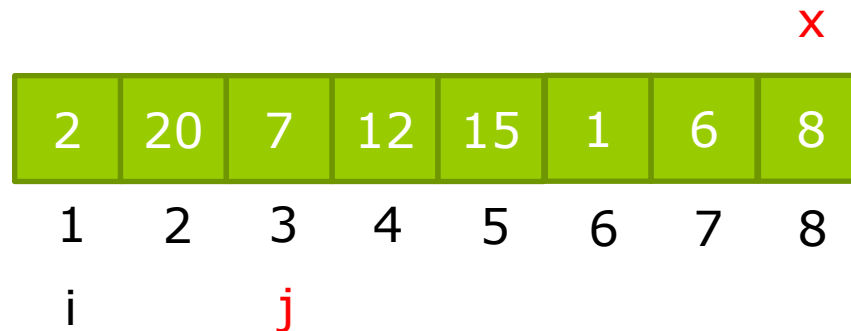
```
1   $x = A[r]$  // the pivot
2   $i = p - 1$  // highest index into the low side
3  for  $j = p$  to  $r - 1$  // process each element other than the pivot
4      if  $A[j] \leq x$  // does this element belong on the low side?
5           $i = i + 1$  // index of a new slot in the low side
6          exchange  $A[i]$  with  $A[j]$  // put this element there
7  exchange  $A[i + 1]$  with  $A[r]$  // pivot goes just to the right of the low side
8  return  $i + 1$  // new index of the pivot
```



Partitioning

PARTITION(A, p, r)

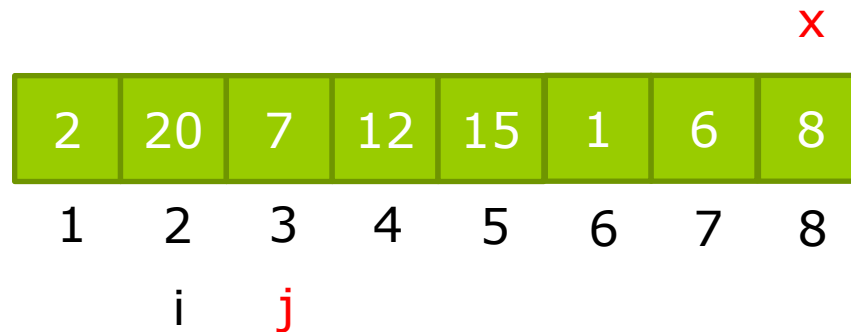
```
1   $x = A[r]$  // the pivot
2   $i = p - 1$  // highest index into the low side
3  for  $j = p$  to  $r - 1$  // process each element other than the pivot
4      if  $A[j] \leq x$  // does this element belong on the low side?
5           $i = i + 1$  // index of a new slot in the low side
6          exchange  $A[i]$  with  $A[j]$  // put this element there
7  exchange  $A[i + 1]$  with  $A[r]$  // pivot goes just to the right of the low side
8  return  $i + 1$  // new index of the pivot
```



Partitioning

PARTITION(A, p, r)

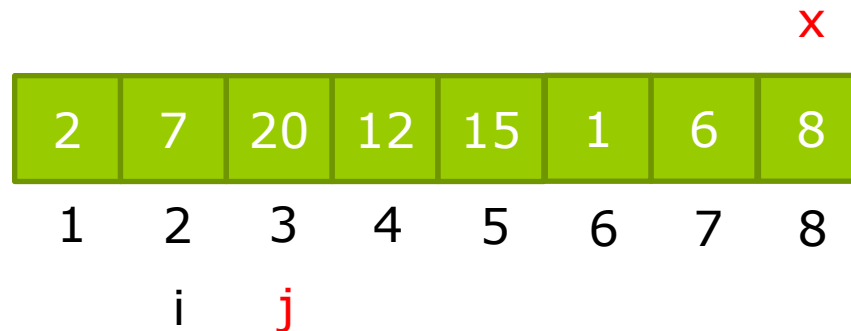
```
1   $x = A[r]$  // the pivot
2   $i = p - 1$  // highest index into the low side
3  for  $j = p$  to  $r - 1$  // process each element other than the pivot
4      if  $A[j] \leq x$  // does this element belong on the low side?
5           $i = i + 1$  // index of a new slot in the low side
6          exchange  $A[i]$  with  $A[j]$  // put this element there
7  exchange  $A[i + 1]$  with  $A[r]$  // pivot goes just to the right of the low side
8  return  $i + 1$  // new index of the pivot
```



Partitioning

PARTITION(A, p, r)

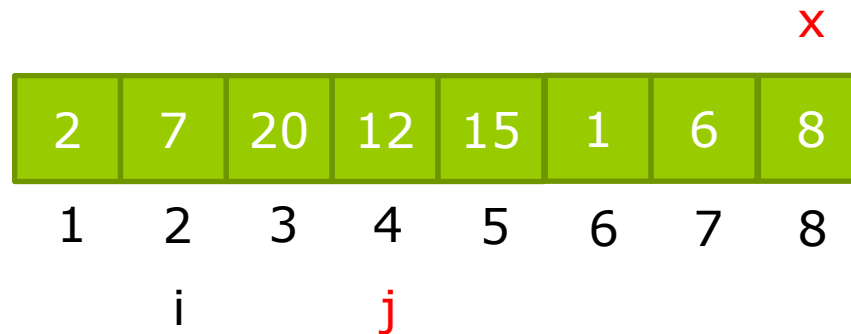
```
1   $x = A[r]$  // the pivot
2   $i = p - 1$  // highest index into the low side
3  for  $j = p$  to  $r - 1$  // process each element other than the pivot
4      if  $A[j] \leq x$  // does this element belong on the low side?
5           $i = i + 1$  // index of a new slot in the low side
6          exchange  $A[i]$  with  $A[j]$  // put this element there
7  exchange  $A[i + 1]$  with  $A[r]$  // pivot goes just to the right of the low side
8  return  $i + 1$  // new index of the pivot
```



Partitioning

PARTITION(A, p, r)

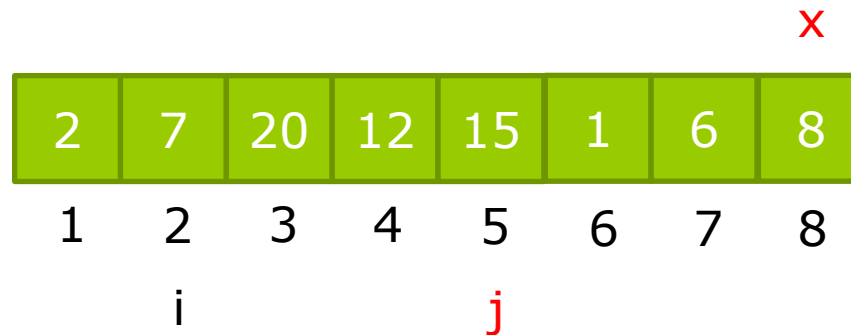
```
1   $x = A[r]$  // the pivot
2   $i = p - 1$  // highest index into the low side
3  for  $j = p$  to  $r - 1$  // process each element other than the pivot
4      if  $A[j] \leq x$  // does this element belong on the low side?
5           $i = i + 1$  // index of a new slot in the low side
6          exchange  $A[i]$  with  $A[j]$  // put this element there
7  exchange  $A[i + 1]$  with  $A[r]$  // pivot goes just to the right of the low side
8  return  $i + 1$  // new index of the pivot
```



Partitioning

PARTITION(A, p, r)

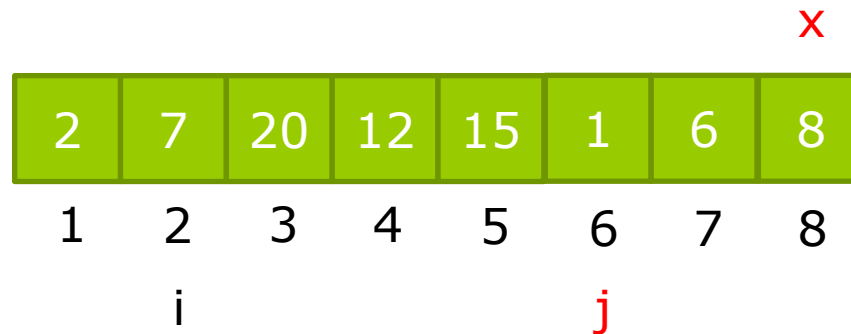
```
1   $x = A[r]$  // the pivot
2   $i = p - 1$  // highest index into the low side
3  for  $j = p$  to  $r - 1$  // process each element other than the pivot
4      if  $A[j] \leq x$  // does this element belong on the low side?
5           $i = i + 1$  // index of a new slot in the low side
6          exchange  $A[i]$  with  $A[j]$  // put this element there
7  exchange  $A[i + 1]$  with  $A[r]$  // pivot goes just to the right of the low side
8  return  $i + 1$  // new index of the pivot
```



Partitioning

PARTITION(A, p, r)

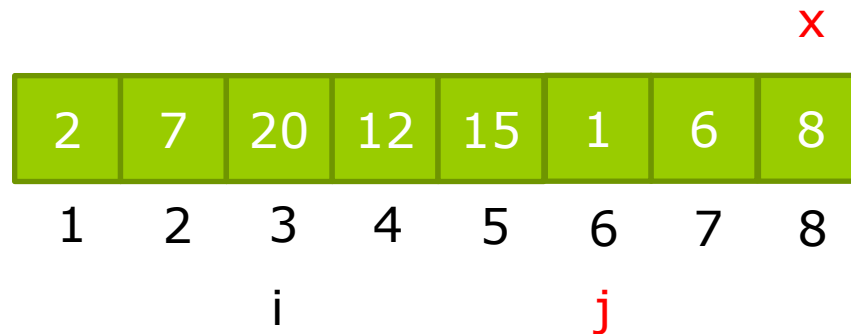
```
1   $x = A[r]$  // the pivot
2   $i = p - 1$  // highest index into the low side
3  for  $j = p$  to  $r - 1$  // process each element other than the pivot
4      if  $A[j] \leq x$  // does this element belong on the low side?
5           $i = i + 1$  // index of a new slot in the low side
6          exchange  $A[i]$  with  $A[j]$  // put this element there
7  exchange  $A[i + 1]$  with  $A[r]$  // pivot goes just to the right of the low side
8  return  $i + 1$  // new index of the pivot
```



Partitioning

PARTITION(A, p, r)

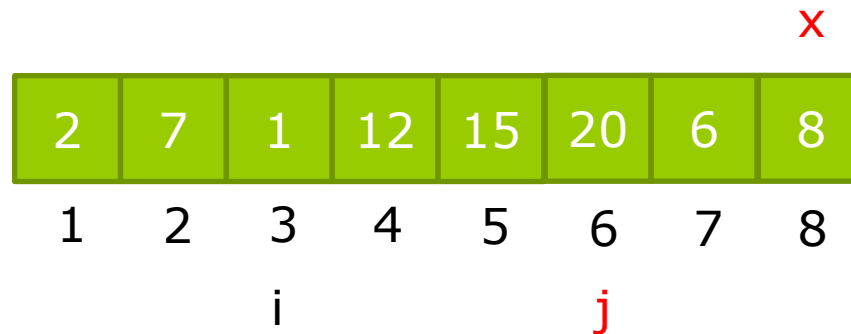
```
1   $x = A[r]$  // the pivot
2   $i = p - 1$  // highest index into the low side
3  for  $j = p$  to  $r - 1$  // process each element other than the pivot
4      if  $A[j] \leq x$  // does this element belong on the low side?
5           $i = i + 1$  // index of a new slot in the low side
6          exchange  $A[i]$  with  $A[j]$  // put this element there
7  exchange  $A[i + 1]$  with  $A[r]$  // pivot goes just to the right of the low side
8  return  $i + 1$  // new index of the pivot
```



Partitioning

PARTITION(A, p, r)

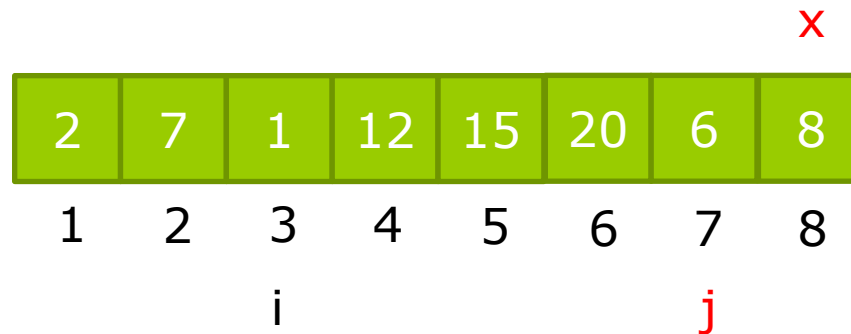
```
1   $x = A[r]$  // the pivot
2   $i = p - 1$  // highest index into the low side
3  for  $j = p$  to  $r - 1$  // process each element other than the pivot
4      if  $A[j] \leq x$  // does this element belong on the low side?
5           $i = i + 1$  // index of a new slot in the low side
6          exchange  $A[i]$  with  $A[j]$  // put this element there
7  exchange  $A[i + 1]$  with  $A[r]$  // pivot goes just to the right of the low side
8  return  $i + 1$  // new index of the pivot
```



Partitioning

PARTITION(A, p, r)

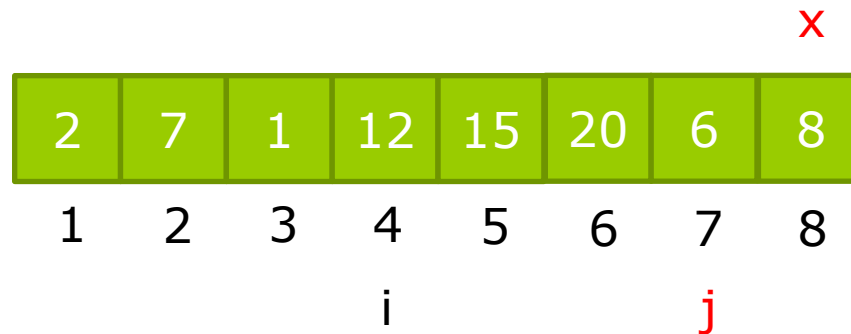
```
1   $x = A[r]$  // the pivot
2   $i = p - 1$  // highest index into the low side
3  for  $j = p$  to  $r - 1$  // process each element other than the pivot
4      if  $A[j] \leq x$  // does this element belong on the low side?
5           $i = i + 1$  // index of a new slot in the low side
6          exchange  $A[i]$  with  $A[j]$  // put this element there
7  exchange  $A[i + 1]$  with  $A[r]$  // pivot goes just to the right of the low side
8  return  $i + 1$  // new index of the pivot
```



Partitioning

PARTITION(A, p, r)

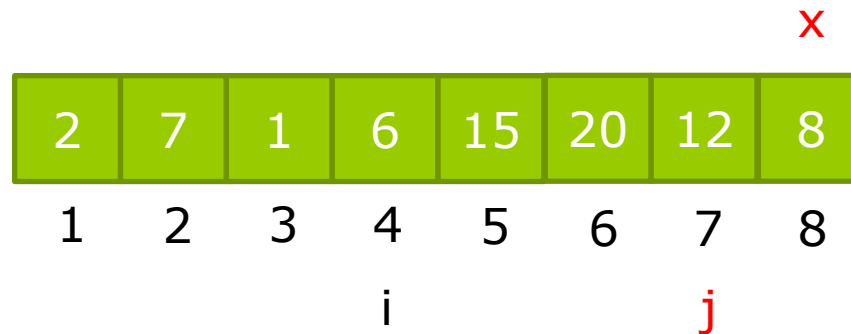
```
1   $x = A[r]$  // the pivot
2   $i = p - 1$  // highest index into the low side
3  for  $j = p$  to  $r - 1$  // process each element other than the pivot
4      if  $A[j] \leq x$  // does this element belong on the low side?
5           $i = i + 1$  // index of a new slot in the low side
6          exchange  $A[i]$  with  $A[j]$  // put this element there
7  exchange  $A[i + 1]$  with  $A[r]$  // pivot goes just to the right of the low side
8  return  $i + 1$  // new index of the pivot
```



Partitioning

PARTITION(A, p, r)

```
1   $x = A[r]$  // the pivot
2   $i = p - 1$  // highest index into the low side
3  for  $j = p$  to  $r - 1$  // process each element other than the pivot
4      if  $A[j] \leq x$  // does this element belong on the low side?
5           $i = i + 1$  // index of a new slot in the low side
6      exchange  $A[i]$  with  $A[j]$  // put this element there
7  exchange  $A[i + 1]$  with  $A[r]$  // pivot goes just to the right of the low side
8  return  $i + 1$  // new index of the pivot
```



Partitioning

PARTITION(A, p, r)

```
1   $x = A[r]$  // the pivot
2   $i = p - 1$  // highest index into the low side
3  for  $j = p$  to  $r - 1$  // process each element other than the pivot
4      if  $A[j] \leq x$  // does this element belong on the low side?
5           $i = i + 1$  // index of a new slot in the low side
6          exchange  $A[i]$  with  $A[j]$  // put this element there
7  exchange  $A[i + 1]$  with  $A[r]$  // pivot goes just to the right of the low side
8  return  $i + 1$  // new index of the pivot
```



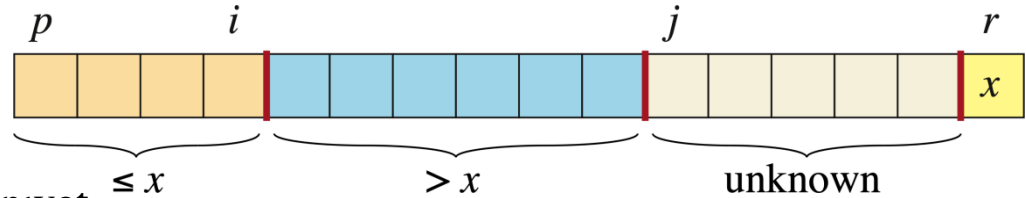
Correctness of PARTITION

□ *Loop invariant:*

1. All entries in $A[p : i]$ are $\leq$ pivot.

2. All entries in $A[i + 1 : j - 1]$ are $>$ pivot.

3. $A[r] = \text{pivot}$.



It is not needed as part of the loop invariant, but the fourth region is $A[p : r]$, whose entries have not yet been examined, and so we do not know how they compare to the pivot.

□ *Correctness*

- **Initialization:** Before the loop starts, all the conditions of the loop invariant are satisfied, because r is the pivot and the subarrays $A[p : i]$ and $A[i + 1 : j - 1]$ are empty.
- **Maintenance:** While the loop is running, if $A[j] \leq \text{pivot}$, then $A[j]$ and $A[i + 1]$ are swapped and then i and j are incremented. If $A[j] > \text{pivot}$, then increment only j .
- **Termination:** When the loop terminates, $j = r$, so that all elements in A are partitioned into one of the three cases: $A[p : i] \leq \text{pivot}$, $A[i + 1 : r - 1] > \text{pivot}$, and $A[r] = \text{pivot}$.

The last two lines of `PARTITION` move the pivot element from the end of the array to between the two subarrays. This is done by swapping the pivot and the first element of the second subarray, i.e., by swapping $A[i + 1]$ and $A[r]$.

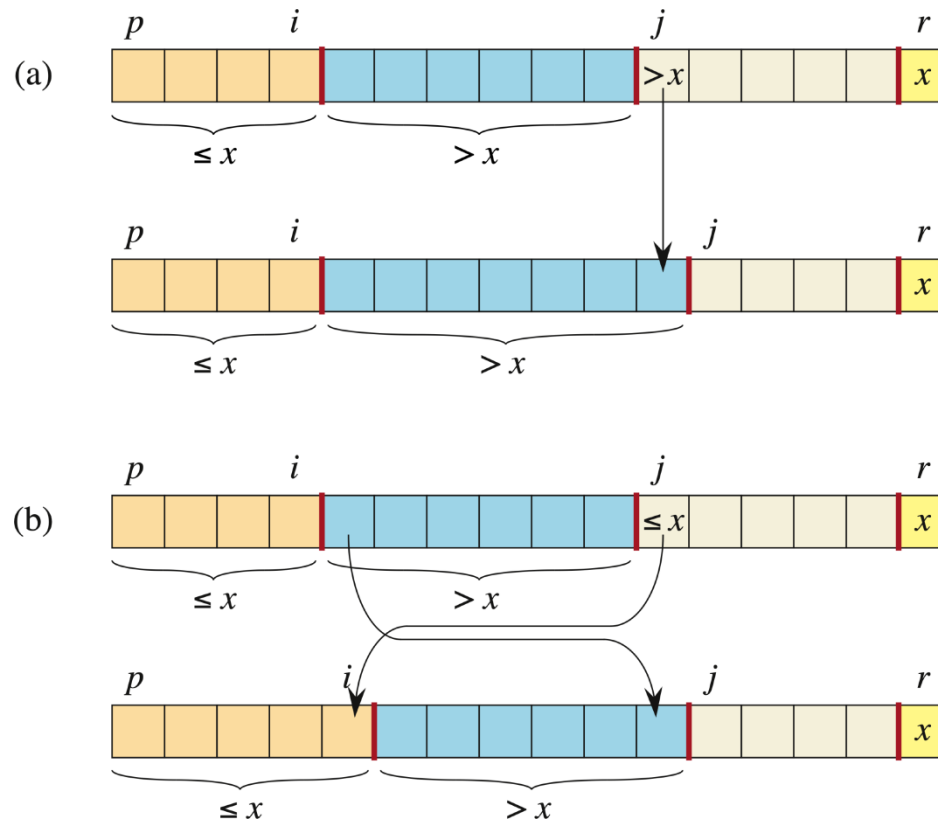
□ *Time for partitioning:* $\Theta(n)$ to partition an n -element subarray.

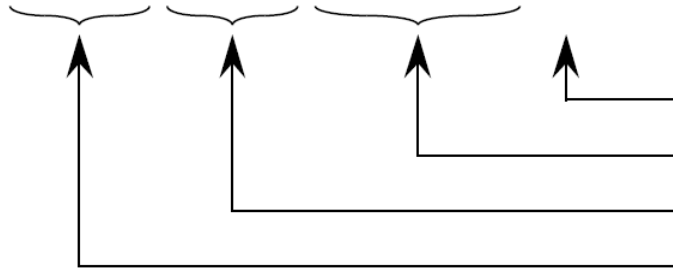
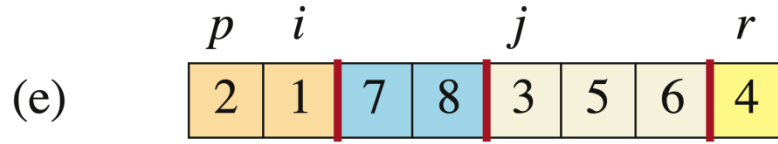
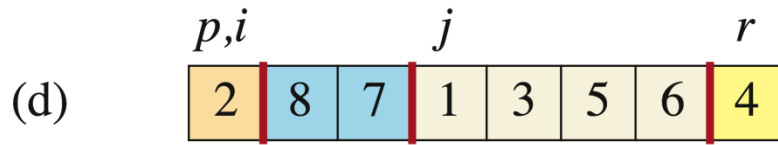
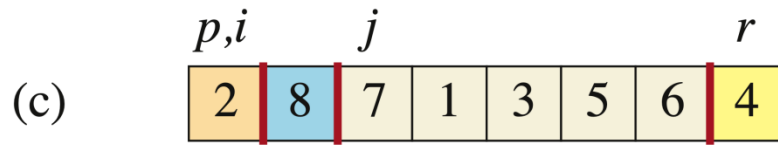
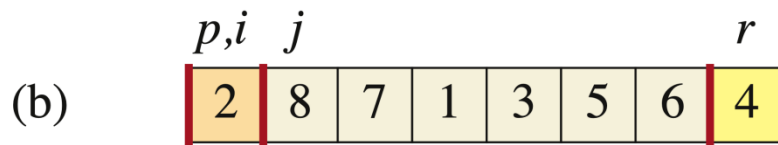
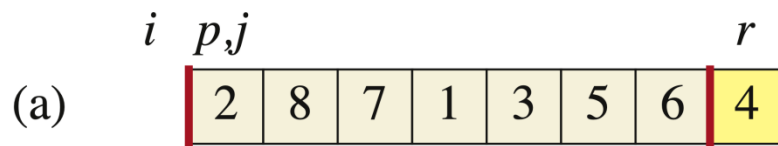
Partitioning

□ The two cases for one iteration of procedure PARTITION

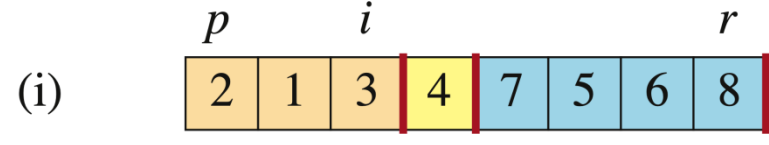
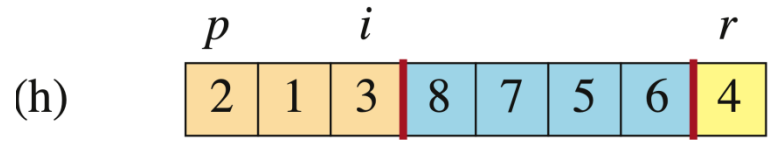
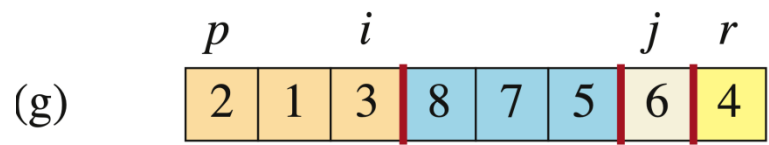
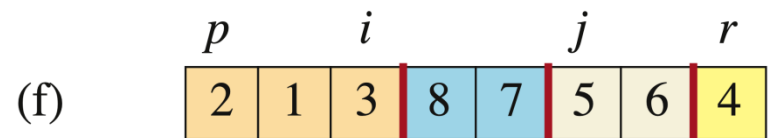
(a) If $A[j] > x$, the only action is to increment j , which maintains the loop invariant.

(b) If $A[j] \leq x$, index i is incremented, $A[i]$ and $A[j]$ are swapped, and j is incremented. Again, the loop invariant is maintained.





$A[r]$: pivot
 $A[j \dots r-1]$: not yet examined
 $A[i+1 \dots j-1]$: known to be $>$ pivot
 $A[p \dots i]$: known to be $\leq$ pivot



Performance of quicksort

- The running time of quicksort depends on the partitioning of the subarrays:
 - If the subarrays are balanced, then quicksort can run as fast as mergesort.
 - If they are unbalanced, then quicksort can run as slowly as insertion sort.

- Worst case
 - Occurs when the subarrays are completely unbalanced.
 - Have 0 elements in one subarray and $n - 1$ elements in the other subarray.
 - Get the recurrence: $T(n) = T(n - 1) + T(0) + \Theta(n) = T(n - 1) + \Theta(n) = \Theta(n^2)$.
 - Same worst-case running time as insertion sort.
 - In fact, the worst-case running time occurs when quicksort takes a sorted array as input, but insertion sort runs in $O(n)$ time in this case.

- Best case
 - Occurs when the subarrays are completely balanced every time.
 - Each subarray has $\leq n/2$ elements.
 - Get the recurrence: $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \lg n)$.

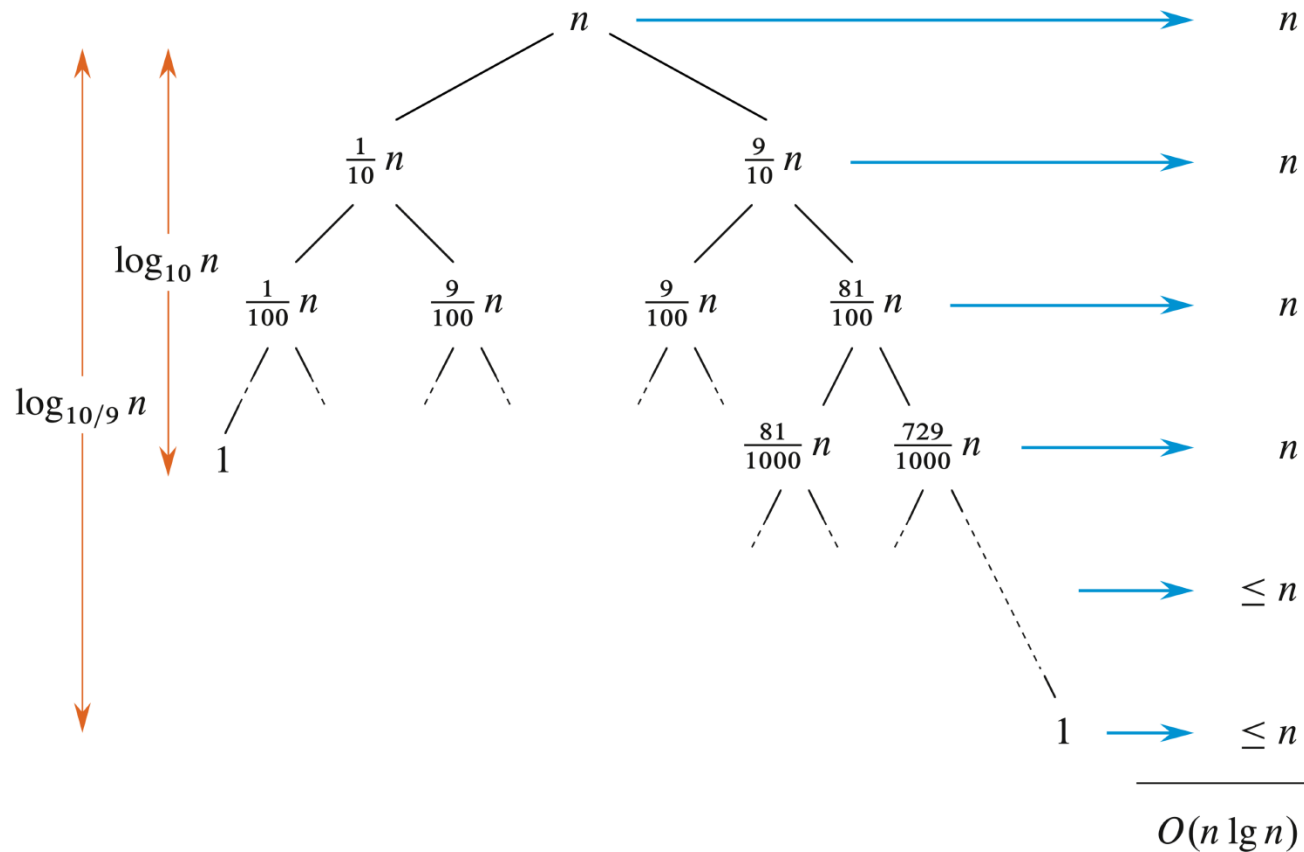
Balanced Partitioning

- ❑ Quicksort's average running time is much closer to the best case than to the worst case.
- ❑ Imagine that PARTITION always produces a 9-to-1 split.
- ❑ Get the recurrence
$$T(n) \leq T(9n/10) + T(n/10) + \Theta(n)$$
$$= O(n \lg n) .$$
- ❑ Intuition: look at the recursion tree.
 - It is like the one for $T(n) = T(n/3) + T(2n/3) + O(n)$ in Lecture 4.
 - Except that here the constants are different: $\log_{10} n$ full levels and $\log_{10/9} n$ levels that are nonempty.
 - As long as it is a constant, the base of the log does not matter in asymptotic notation.
 - Any split of constant proportionality will yield a recursion tree of depth $\Theta(\lg n)$.

Balanced Partitioning

- The recursion tree for the recurrence

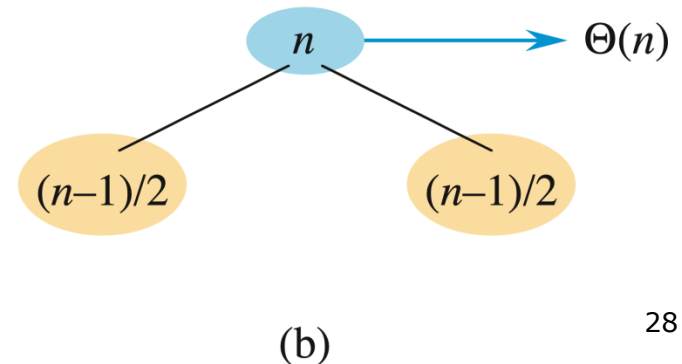
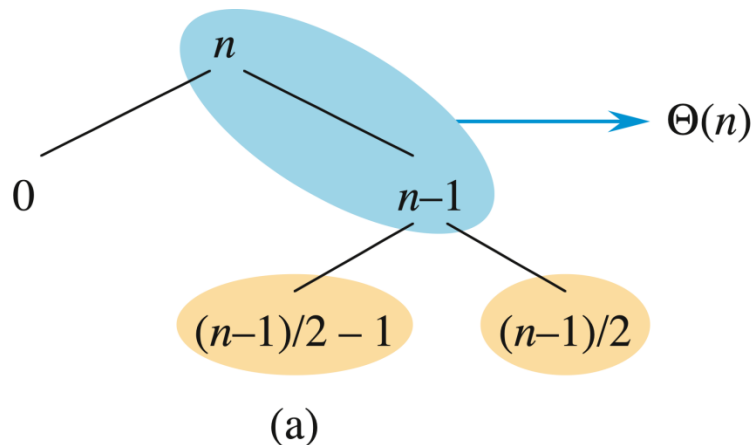
$$T(n) = T(9n/10) + T(n/10) + \Theta(n)$$



Intuition for the average case

□ In the following figure

- (a) Two levels of a recursion tree for quicksort. The partitioning at the root costs n and produces a “bad” split: two subarrays of size 0 and $n - 1$. The partitioning of the subarray of size $n - 1$ costs $n - 1$ and produces a “good” split: subarrays of size $(n - 1)/2 - 1$ and $(n - 1)/2$.
- (b) A single level of a recursion tree that is well balanced. In both parts, the partitioning cost for the subproblems shown with blue shading is $\Theta(n)$. Yet the subproblems remaining to be solved in (a), shown with tan shading, are no larger than the corresponding subproblems remaining to be solved in (b).



Intuition for the average case

- ❑ In the average case, PARTITION produces a mix of good and bad splits, which are distributed randomly throughout the tree.
- ❑ Suppose that the good and bad splits alternate levels in the tree, and that the good splits are best-case splits and the bad splits are worst-case splits.
- ❑ The combination of the bad split followed by the good split produces three subarrays of sizes 0, $(n - 1)/2 - 1$, and $(n - 1)/2$ at a combined partitioning cost of $\Theta(n) + \Theta(n - 1) = \Theta(n)$. This situation is at most a constant factor worst than that in Figure (b), namely, where a single level of partitioning produces two subarrays of size $(n - 1)/2$, at a cost of $\Theta(n)$. Yet this latter situation is balanced!
- ❑ Intuitively, the $\Theta(n - 1)$ cost of the bad split can be absorbed into the $\Theta(n)$ cost of good split, and the resulting split is good.
- ❑ Thus, the running time of quicksort, when levels alternate between good and bad splits, is like the running time for good splits alone: still $O(n \lg n)$, but with a slightly larger constant hidden by the O -notation.

Randomized version of quicksort

- We have assumed that all input permutations are equally likely. This is not always true.
- To correct this, we add randomization to quicksort. We could randomly permute the input array.
- Instead, we use *random sampling*, or picking one element at random.
- Do not always use $A[r]$ as the pivot. Instead, randomly pick an element from the subarray that is being sorted. It causes the split of the input array to be reasonably well balanced. Thus randomization of quicksort stops any specific type of array from causing worst-case behavior.

RANDOMIZED-PARTITION(A, p, r)

```
1  $i = \text{RANDOM}(p, r)$ 
2 exchange  $A[r]$  with  $A[i]$ 
3 return PARTITION( $A, p, r$ )
```

RANDOMIZED-QUICKSORT(A, p, r)

```
1 if  $p < r$ 
2      $q = \text{RANDOMIZED-PARTITION}(A, p, r)$ 
3     RANDOMIZED-QUICKSORT( $A, p, q - 1$ )
4     RANDOMIZED-QUICKSORT( $A, q + 1, r$ )
```

Analysis of quicksort: Worst case

- We will prove that a worst-case split at every level in QUICKSORT or in RANDOMIZED-QUICKSORT produces a worst-case running time of $O(n^2)$.
 - Recurrence for the worst-case running time of QUICKSORT or RANDOMIZED-QUICKSORT (the same):
$$T(n) = \max \{T(q) + T(n - 1 - q) : 0 \leq q \leq n - 1\} + \Theta(n) .$$
 - Because PARTITION produces two subproblems, totaling size $n - 1$, q ranges from 0 to $n - 1$.
 - **Guess:** $T(n) \leq cn^2$, for some c .
 - Substituting our guess into the above recurrence:
$$\begin{aligned} T(n) &\leq \max \{cq^2 + c(n - 1 - q)^2 : 0 \leq q \leq n - 1\} + \Theta(n) \\ &= c \cdot \max \{q^2 + (n - 1 - q)^2 : 0 \leq q \leq n - 1\} + \Theta(n) . \end{aligned}$$
 - Let us focus our attention on the maximization. For $q = 0, 1, \dots, n - 1$, we have
$$\begin{aligned} q^2 + (n - 1 - q)^2 &= q^2 + (n - 1)^2 - 2q(n - 1) + q^2 \\ &= (n - 1)^2 + 2q(q - (n - 1)) \\ &\leq (n - 1)^2 \end{aligned}$$
because $q \leq n - 1$ implies that $2q(q - (n - 1)) \leq 0$. Thus every term in the maximization is bound by $(n - 1)^2$.
 - And thus, $T(n) \leq c(n - 1)^2 + \Theta(n) \leq cn^2 - c(2n - 1) + \Theta(n) \leq cn^2$ by picking the constant c large enough that the $c(2n - 1)$ term dominates the $\Theta(n)$ term.
 - Therefore, the worst-case running time of quicksort is $O(n^2)$.
 - Can also show that the recurrence's solution is $\Omega(n^2)$. Thus, the worst-case running time is $\Theta(n^2)$.

Analysis of quicksort: Expected running time

- Assume that all values being sorted are distinct.
- The dominant cost of the algorithm is partitioning.
- Assume that RANDOMIZED-PARTITION makes it so that the pivot selected by PARTITION is selected randomly from the subarray passed to these procedures.
- PARTITION removes the pivot element from future consideration each time.
- Thus, PARTITION is called at most n times.
- QUICKSORT recurses on the partitions.
- The amount of work that each call to PARTITION does is a constant plus the number of comparisons that are performed in its **for** loop.
- Let X = the total number of comparisons performed in all calls to PARTITION.
- Therefore, the total work done over the entire execution is $O(n + X)$.

Analysis of quicksort: Expected running time

- Need to compute a bound on the overall number of comparisons.
- For ease of analysis:
 - Rename the elements of A as $z_1, z_2, \dots, z_n$, with z_i being the i th smallest element. (So that the output order is $z_1, z_2, \dots, z_n$.)
 - Define the set $Z_{i,j} = \{z_i, z_{i+1}, \dots, z_j\}$ to be the set of elements between z_i and z_j , inclusive.
- Each pair of elements is compared at most once, because elements are compared only with the pivot element, and then the pivot element is never in any later call to PARTITION.
 - As an example, consider an input to quicksort of the numbers 1 through 10 in some arbitrary order. Suppose that the first pivot element is 7. Then the first call to PARTITION separates the numbers into two sets: $\{1, 2, 3, 4, 5, 6\}$ and $\{8, 9, 10\}$.
 - In the process, the pivot element 7 is compared with all other elements, but no number from the first set (e.g., 2) is or ever will be compared with any number from the second set (e.g., 9).
 - The values 7 and 9 are compared because 7 is the first item from $Z_{7,9}$ to be chosen as a pivot. In contrast, 2 and 9 are never compared because the first pivot element chosen from $Z_{2,9}$ is 7.

Analysis of quicksort: Expected running time

- Let $X_{ij} = \mathbf{I} \{z_i \text{ is compared with } z_j\}$
(Considering whether z_i is compared with z_j at any time during the entire quicksort algorithm, not just during one call of PARTITION.)
- Since each pair is compared at most once, the total number of comparisons performed by the algorithm is

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij} .$$

- Take expectations of both sides, use Lemma 5.1 and linearity of expectations:

$$\begin{aligned} E[X] &= E \left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij} \right] \\ &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}] && \text{(by linearity of expectation)} \\ &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \Pr \{z_i \text{ is compared with } z_j\} && \text{(by Lemma 5.1)} \end{aligned}$$

Analysis of quicksort: Expected running time

- Now all we have to do is to find the probability that two elements are compared.
 - Think about when two elements are *not* compared.
 - Once a pivot x is chosen such that $z_i < x < z_j$, then z_i and z_j will never be compared at any later time.
 - If either z_i or z_j is chosen as the first pivot before any other element of $Z_{i,j}$, then it will be compared with all the elements of $Z_{i,j}$, except itself.
 - The probability that z_i is compared with z_j is the probability that either z_i or z_j is the first element chosen.
 - There are $j - i + 1$ elements, and pivots are chosen randomly and independently. Thus, the probability that any particular one of them is the first one chosen is $1/(j - i + 1)$.
 - The event that z_i is the first pivot and the event that z_j is the first pivot are mutually exclusive.

Analysis of quicksort: Expected running time

□ Therefore,

$$\begin{aligned}\Pr\{z_i \text{ is compared with } z_j\} &= \Pr\{z_i \text{ or } z_j \text{ is the first pivot chosen from } Z_{ij}\} \\ &= \Pr\{z_i \text{ is the first pivot chosen from } Z_{ij}\} \\ &\quad + \Pr\{z_j \text{ is the first pivot chosen from } Z_{ij}\} \\ &= \frac{1}{j-i+1} + \frac{1}{j-i+1} \\ &= \frac{2}{j-i+1}.\end{aligned}$$

□ Substituting into the equation for $E[X]$:

$$E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1}.$$

Analysis of quicksort: Average case

- Evaluate by using a change in variable ($k = j - i$) and the bound on the harmonic series in equation (A.9):

$$\begin{aligned} E[X] &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1} \\ &= \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} \\ &< \sum_{i=1}^{n-1} \sum_{k=1}^n \frac{2}{k} \\ &= \sum_{i=1}^{n-1} O(\lg n) \\ &= O(n \lg n) . \end{aligned}$$

- So the expected running time of quicksort, using RANDOMIZED-PARTITION, is $O(n \lg n)$ if all values being sorted are distinct.

Reading

- ▣ Sections 7.1 ~ 7.4